



Greedy Algorithm



Definition

A greedy algorithm is a problem-solving technique used in computer science and mathematics. It involves making the **locally optimal** choice at each stage of a problem with the **hope of finding a global optimum solution**. In other words, at each step of the algorithm, it selects the best possible choice without considering the future consequences. The algorithm continues making these choices until it reaches the end, aiming to find the best overall solution.



Coin Change problem

Given a set of coins with different denominations and a target amount, find the minimum number of coins required to make up that amount.



Solution

- Start with the highest denomination coin.
- Choose the largest possible number of that coin that does not exceed the target amount.
- Subtract the value of the chosen coins from the target amount.
- Repeat steps 1-3 until the target amount becomes 0.



Example 1

Let's say we have coins with denominations $[1, 5, 10, 25]$ (in cents) and we want to make a change for 63 cents.

```

int findClosestCoinValue(int coin[], int currentAmount, int numberOfCoin) {
    int diff = INT_MAX;

    int closestCoinValue = INT_MAX;
    for(int i = 0; i<numberOfCoin; i++) {
        int tempDifference = currentAmount - coin[i];
        if(tempDifference >= 0 && tempDifference < diff) {
            diff = tempDifference;
            closestCoinValue = coin[i];
        }
    }
    return closestCoinValue;
}

int main() {
    int coin[4]{1,5,10,25};
    int numberOfCoin = sizeof(coin) / sizeof(coin[0]);
    int targetAmount = 63;
    int currentAmount = targetAmount;

    while (currentAmount >0) {
        int closestCoinValue = findClosestCoinValue(coin, currentAmount, numberOfCoin);
        currentAmount -= closestCoinValue;
        cout << closestCoinValue << " ";
    }

    cout << endl;
}

```



Example 2

Let's say we have coins with denominations $[1, 3, 4]$ (in cents) and we want to make a change for 6 cents.



Greedy-choice property

The greedy-choice property is a property of certain optimization problems, particularly those that can be solved using greedy algorithms. It states that at each step of the problem-solving process, making the locally optimal choice leads to an overall optimal solution.

In other words, the greedy-choice property implies that if we consistently choose the best immediate option available without considering the future consequences, we will eventually reach the best possible solution for the entire problem.



Optimal substructure

Optimal substructure refers to a property of certain problems where an optimal solution to the overall problem can be constructed from optimal solutions to its subproblems.

More formally, a problem exhibits optimal substructure if an optimal solution to the problem contains within it optimal solutions to subproblems.



Activity Selection Problem

Given a set of activities, each with a start time and finish time, the objective is to select the maximum number of non-overlapping activities that can be performed.



Activity Selection Problem

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16



Activity Selection Problem

The activity selection problem exhibits both the greedy-choice property and optimal substructure, making it amenable to a greedy algorithm that provides an optimal solution



Activity Selection Problem

Greedy-choice property: At each step of the greedy algorithm, selecting the activity with the earliest finish time ensures that we maximize the remaining time available for selecting subsequent activities. By choosing the activity that finishes earliest, we free up time for more activities to be performed later. This locally optimal choice contributes to the overall optimal solution.



Activity Selection Problem

Optimal substructure: The problem has the property that once we've made a locally optimal choice (selecting an activity with the earliest finish time), the subproblem that remains is still an instance of the original problem. This means that the optimal solution to the entire problem can be constructed from the optimal solutions to its subproblems. In other words, if we have an optimal solution for a smaller set of activities, we can extend it to include the current activity (with the earliest finish time) to obtain an optimal solution for the larger set.



Knapsack problem

The knapsack problem is a classic optimization problem where given a set of items, each with a weight and a value, the goal is to determine the maximum value that can be obtained by selecting a subset of the items such that the total weight does not exceed a given limit (the capacity of the knapsack).



Fractional Knapsack problem

There is a variant of the knapsack problem called the fractional knapsack problem, where items can be divided into fractions and included in the knapsack. In this case, a greedy algorithm can be used to find the optimal solution.



Does greedy algorithm give optimal solution to 0-1 knapsack problem?



Example

Suppose we have a knapsack with a capacity of 50 units and the following items:

- Item 1: Weight = 20 units, Value = \$100
- Item 2: Weight = 30 units, Value = \$120
- Item 3: Weight = 10 units, Value = \$60



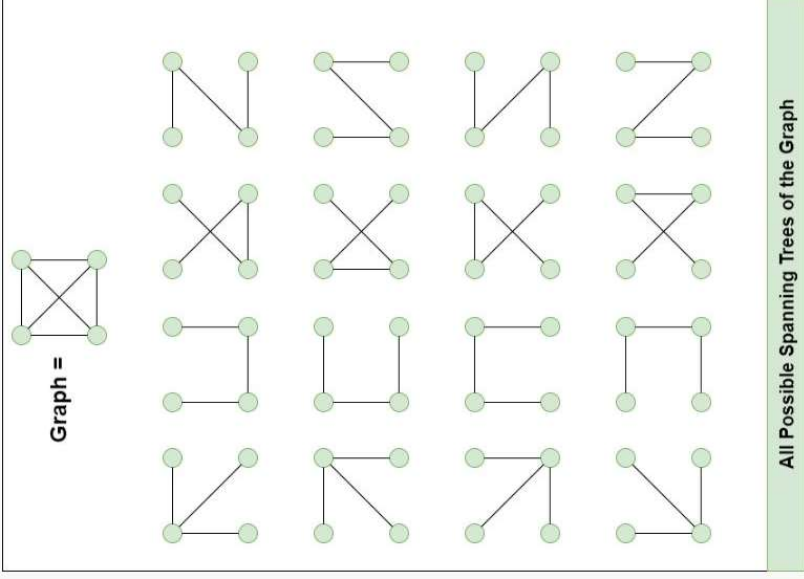
Spanning tree

A spanning tree of a connected, undirected graph is a subgraph that is a tree and connects all the vertices together without forming any cycles. In other words, it's a tree that spans all the vertices of the original graph.



Spanning tree

Source: [geeksforgeeks](https://www.geeksforgeeks.org/)



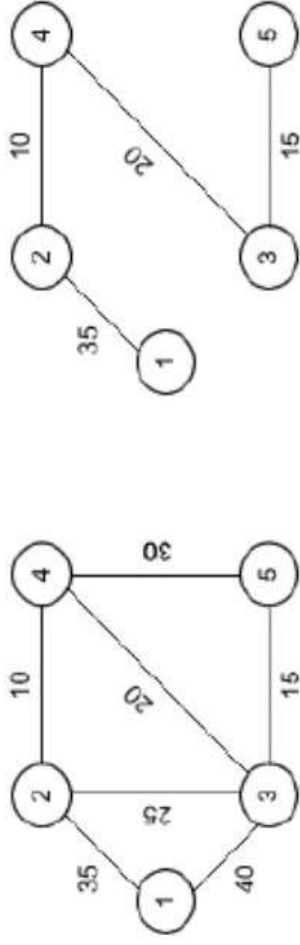


Minimum Spanning Tree

A minimum spanning tree (MST) of a connected, undirected graph is a spanning tree with the smallest possible sum of edge weights. This means that an MST connects all vertices of the graph with the minimum total edge weight possible.



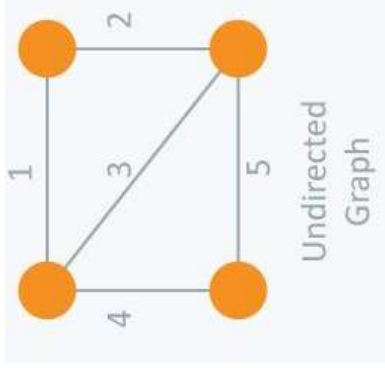
Minimum Spanning Tree



Source: researchgate



Find MST





How to Find Minimum Spanning Tree?

We can find the MST using a greedy algorithm called Kruskal's algorithm or Prim's algorithm.



Kruskal's Algorithm

1. Sort all the edges in non-decreasing order of their weight.
2. Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If the cycle is not formed, include this edge. Else, discard it.
3. Repeat step#2 until there are $(V-1)$ edges in the spanning tree.



Prim's Algorithm

Step 1: Determine an arbitrary vertex as the starting vertex of the MST.

Step 2: Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).

Step 3: Find edges connecting any tree vertex with the fringe vertices.

Step 4: Find the minimum among these edges.

Step 5: Add the chosen edge to the MST if it does not form any cycle.

Step 6: Return the MST and exit



Example

